

Aether — KubeVirt VMs

Virtual Machine Deployment on Kubernetes

Run full virtual machines alongside containers on your Kubernetes cluster.
Aether generates VirtualMachine CRDs and DataVolumes from the same
workload spec used for containers.

VirtualMachine CRD • DataVolume • GPU Passthrough • Live Migration

What is KubeVirt?

KubeVirt extends Kubernetes with virtualization capabilities, allowing VMs and containers to coexist in the same cluster under the same management plane.



VMs on Kubernetes

KubeVirt adds a VirtualMachine CRD to Kubernetes, letting you define, schedule, and manage VMs just like pods. The kubelet handles VM lifecycle through the virt-handler DaemonSet.



Unified Scheduling

VMs are scheduled by the Kubernetes scheduler alongside containers. Resource requests, node affinity, taints, and tolerations all work the same way for both workload types.



Full VM Isolation

Unlike containers, KubeVirt VMs provide full hardware-level isolation via KVM. Ideal for legacy applications, compliance-sensitive workloads, and multi-tenant environments.



Kubernetes Networking

VMs connect to the same CNI network as pods. Services, Ingress, and NetworkPolicies work transparently, enabling VMs and containers to communicate natively.

KubeVirt is the bridge between legacy VM workloads and modern Kubernetes infrastructure — and Aether makes deploying to it as simple as deploying a container.

How Aether Deploys VMs

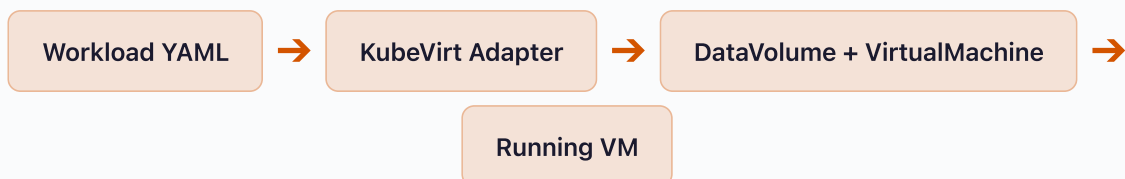
Aether's KubeVirt adapter generates two CRDs from your workload spec: a DataVolume for the disk image and a VirtualMachine for the VM definition.

DataVolume (Disk Image)

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: my-vm-disk
  namespace: default
  labels:
    app: my-vm
    managed-by: aether
spec:
  source:
    registry:
      url: "docker://quay.io/containerdisks/fedora:40"
  pvc:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 20Gi
```

VirtualMachine CRD

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
  namespace: default
  labels:
    app: my-vm
    managed-by: aether
spec:
  running: true
  template:
    spec:
      domain:
        cpu:
          cores: 4
        resources:
          requests:
            memory: 8Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
      volumes:
        - name: rootdisk
          dataVolume:
            name: my-vm-disk
```



GPU Passthrough

Aether maps GPU requirements from your workload spec directly to KubeVirt GPU device passthrough, enabling ML training and inference workloads inside VMs.

Workload Spec with GPU

```
apiVersion: aether/v1
kind: Workload
metadata:
  name: ml-training-vm
spec:
  requirements:
    cpu: "8"
    memory: 32Gi
    storage: 100Gi
    gpu:
      count: 2
      vendor: nvidia
  runtime:
    preferred: kubevirt
    allow: [kubevirt]
  build:
    context: .
    dockerfile: Dockerfile
    registry: registry.example.com
```

Generated VM with GPU

```
spec:
  template:
    spec:
      domain:
        cpu:
          cores: 8
        resources:
          requests:
            memory: 32Gi
        devices:
          gpus:
            - name: gpu-0
              deviceName: nvidia.com/GP100GL
            - name: gpu-1
              deviceName: nvidia.com/GP100GL
        disks:
          - name: rootdisk
            disk:
              bus: virtio
        volumes:
          - name: rootdisk
            dataVolume:
              name: ml-training-vm-disk
```

NVIDIA

GPU Vendor Support

AMD

GPU Vendor Support

Intel

GPU Vendor Support

Migration & Monitoring

Migrate workloads from containers to VMs with a single command and monitor VMs using the same Aether interface.

Container to VM Migration

```
# Migrate a running container workload to a KubeVirt VM
$ aether migrate my-app kubevirt --strategy blue-green

Migration: my-app [podman -> kubevirt]
Strategy:  blue-green

Step 1/4: Generating DataVolume from container image...
  DataVolume my-app-disk created (20Gi, ReadWriteOnce)

Step 2/4: Creating VirtualMachine CRD...
  VirtualMachine my-app created (4 CPU, 8Gi RAM)

Step 3/4: Waiting for VM to boot and pass health checks...
  VM booted in 45s | Health check: HTTP GET / -> 200 OK

Step 4/4: Switching traffic and cleaning up source...
  Source container stopped
  Migration complete

Status: my-app running on kubevirt (blue-green, 0 downtime)
```

VM Monitoring

Status & Health

```
$ aether status my-vm
```

NAME	RUNTIME	STATE	READY
my-vm	kubevirt	Running	true

```
$ aether health my-vm
Uptime: 99.99% (1440 checks)
Last state: Running
Restarts: 0
Latency: 12ms avg
```

Logs via Serial Console

```
# Stream VM serial console output
$ aether logs my-vm --follow
[ 0.000000] Linux version 6.8.0
[ 1.234567] systemd[1]: Started
[ 2.345678] sshd[456]: Server listening
[ 3.456789] my-app[789]: Ready on :8080

# Access VM via BMC/serial for debugging
$ aether exec my-vm -- virsh console my-vm
```


Aether treats VMs as first-class workloads. The same status, logs, health, and migrate commands work identically whether the target is a container or a virtual machine.

VMs Meet **Kubernetes**

Deploy virtual machines with the same simplicity as containers. One spec, one command, full VM isolation.

Deploy Your First VM

```
# 1. Create a VM workload spec
$ cat workload.yaml
apiVersion: aether/v1
kind: Workload
metadata:
  name: legacy-app
spec:
  requirements:
    cpu: "4"
    memory: 8Gi
    storage: 50Gi
  runtime:
    preferred: kubevirt
    allow: [kubevirt]
  build:
    context: .
    dockerfile: Dockerfile
    registry: registry.example.com

# 2. Deploy the VM
$ aether run workload.yaml
Creating DataVolume: legacy-app-disk (50Gi) ... done
Creating VirtualMachine: legacy-app ... done
Waiting for VM boot ... ready (38s)

Workload legacy-app deployed to kubevirt
```

2

CRDs Generated

KVM

Isolation Level

3

GPU Vendors

Zero

Downtime Migration

From Containers to VMs — Seamlessly

Aether's KubeVirt adapter brings full VM isolation to your Kubernetes cluster without changing your workflow. Same spec format, same CLI, same observability — just stronger isolation.

